

DOCUMENTAÇÃO TÉCNICA DE PROJETO

ThreatLens AI

Modelagem de Ameaças Automatizada

Detecção supervisionada de componentes (YOLOv8n) · Reconstrução de fluxos e fronteiras (OpenCV + OCR) · Quality Gate estrutural fail-closed · Motor STRIDE determinístico · RAG local de contramedidas · Relatórios em JSON, Markdown e PDF.

DESENVOLVEDORA

Luana Caldas

INSTITUIÇÃO

FIAP — Hackathon
Pós-Tech

DATA

Julho de 2026

VERSÃO

1.0 — Produção

Links do Projeto

Acesso direto ao código-fonte versionado e à demonstração em vídeo do ThreatLens AI.

VÍDEO DE DEMONSTRAÇÃO · YOUTUBE

<https://www.youtube.com/watch?v=jSh-K-leQz8&t=5s>

REPOSITÓRIO · GITHUB

<https://github.com/luanacaldas/tc-iadt-fase5-threatlens-stride>

Cabeçalho do documento

Projeto	MVP de Modelagem de Ameaças Automatizada a partir de Diagramas de Arquitetura de Software
Evento	Hackathon Pós-Tech FIAP — Fase 5
Empresa Desafiante	FIAP Software Security
Data de Entrega	Julho de 2026
Repositório GitHub	github.com/luanacaldas/tc-iadt-fase5-threatlens-stride
Vídeo de Demonstração	youtube.com/watch?v=jSh-K-leQz8&t=5s
Status Geral	✓ MVP Validado — 7 de 7 Requisitos Atendidos

Índice

01 Matriz de Conformidade e Rastreabilidade

02 Pipeline Técnico de IA e Visão Computacional

03 Mapeamento STRIDE e Contramedidas

04 Pipeline de Geração de Relatório

05 Limitações Técnicas e Engenharia de Produção

06 Segurança e Conformidade

07 Entregáveis Finais

08 Conclusão

09 Contrato da API

10 Desenvolvimento do Modelo Supervisionado

11 Avaliação do MVP

12 Arquitetura — Detalhamento Complementar

13 Evidências para Entrega e Demonstração

MATRIZ DE CONFORMIDADE E RASTREABILIDADE DE REQUISITOS

A tabela abaixo mapeia cada requisito oficialmente solicitado no edital do Hackathon com o status de implementação, localização no código e evidência técnica.

Requisito do Edital	Status	Evidência e Implementação no Projeto
1. Detecção e Leitura do Diagrama	✓ Atendido	Pipeline de Análise: A entrada do sistema é uma imagem de diagrama de arquitetura. O fluxo começa em <code>backend/main.py</code> (função <code>analyze_image_and_detect()</code> e <code>POST /analyze/image</code>), que valida a imagem antes de processamento. A detecção de componentes é feita por <code>backend/detector.py</code> , que carrega o modelo registrado em <code>models/threatlens-hybrid-v2/weights/best.pt</code> (YOLOv8n treinado). O OCR local é implementado em <code>backend/ocr.py</code> usando Tesseract para extrair labels, nomes de serviços e protocolos. A reconstrução de fluxos e fronteiras acontece em <code>backend/diagram_structure.py</code> usando OpenCV para análise de linhas, conectores e setas. Resultado: cada componente retorna id, nome, tipo, provider, confiança (0.0-1.0) e bounding box (x, y, w, h).
2. Dataset de Arquiteturas de Software	✓ Atendido	Dataset Híbrido Registrado: O dataset final é <code>dataset/hybrid_v2</code> , contendo 373 imagens e 3.048 objetos anotados em 14 classes canônicas. Origem: 300 diagramas do projeto + 73 selecionados do Software Architecture Dataset (Kaggle, 8.700 imagens originais). Splits: 265 treino, 69 validação, 39 teste. Configuração oficializada em <code>dataset/architecture.yaml</code> com <code>nc=14</code> e mapeamento de nomes de classes. Diversidade: inclui diagramas de AWS, Azure, GCP e estilos genéricos. Sem vazamento entre splits — a agrupação por arquitetura original garantiu que variantes aumentadas não cruzassem treino/validação/teste.

Requisito do Edital	Status	Evidência e Implementação no Projeto
3. Anotação de Dados (Rótulos/Labels)	✓ Atendido	Processo de Anotação Auditável: O dataset Kaggle vinha em formato Pascal VOC (.xml). Conversão para YOLO (.txt) em <code>scripts/build_dataset.py</code> com mapeamento para a taxonomia canônica (14 classes). O processo de auditoria de anotações está em <code>scripts/audit_voc_annotations.py</code>, que verifica orfandade, integridade e balanceamento. Dados reais derivados de 9 diagramas de desenvolvimento e 6 blind benchmark foram anotados com <code>scripts/bootstrap_real_benchmark_annotations.py</code>, que registra <code>annotationStatus: "draft_model_proposal"</code> e exige revisão visual explícita (<code>annotationMethod: "YOLO and geometric bootstrap; requires visual review before scoring."</code>). Rastreabilidade completa: cada imagem tem metadados de origem, método e status de validação.
4. Treinamento do Modelo Supervisionado	✓ Atendido	Treinamento YOLOv8n com Reprodutibilidade: Script de treino em <code>scripts/train_yolo.py</code> implementa: (a) carregamento do dataset via <code>dataset/architecture.yaml</code>, (b) uso de YOLOv8n (nano) para eficiência, (c) otimizador AdamW conforme README, (d) resolução 640, (e) replay balanceado entre fontes Kaggle e projeto (55+55 diagramas), (f) validação balanceada (9+9 imagens), (g) epochs configurável (padrão 100), (h) checkpoint automático do melhor peso em <code>models/threatlens-hybrid-v2/weights/best.pt</code>. Hash SHA-256 do peso registrado: <code>2aebcd0611927505d60a10d23ce11d0e5211cf2e91ca0c4394921fac006c865d</code>. Model card completo em <code>models/threatlens-hybrid-v2/model-card.json</code> com métricas, data, versão e hiperparâmetros.

Requisito do Edital	Status	Evidência e Implementação no Projeto
5. Mapeamento de Ameaças STRIDE e Contramedidas Associadas	✓ Atendido	Engine STRIDE Determinístico: <code>backend/stride_engine.py</code> implementa um catálogo de regras rastreáveis por tipo de componente (user, internet, api_gateway, load_balancer, waf, compute, database, storage, queue, identity_provider, monitoring, backup, secrets_kms, cdn). Para cada tipo, há um dicionário <code>COMPONENT_KNOWLEDGE</code> com regras STRIDE (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege). Cada ameaça tem: (a) STRIDE category, (b) title, (c) severity (Critical/High/Medium/Low), (d) confiança calculada a partir de detecção, (e) contramedidas objetivas (lista de ações), (f) referências a CWE (Common Weakness Enumeration), CAPEC (Common Attack Pattern Expression Language), OWASP Top 10 2021 e MITRE ATT&CK. Exemplo: Spoofing em <code>api_gateway</code> → CWE-287 (Improper Authentication), CAPEC-115 (Authentication Bypass), OWASP A07:2021, MITRE T1078 (Valid Accounts). Todas as 6 categorias STRIDE cobertas com referências de segurança verificáveis.

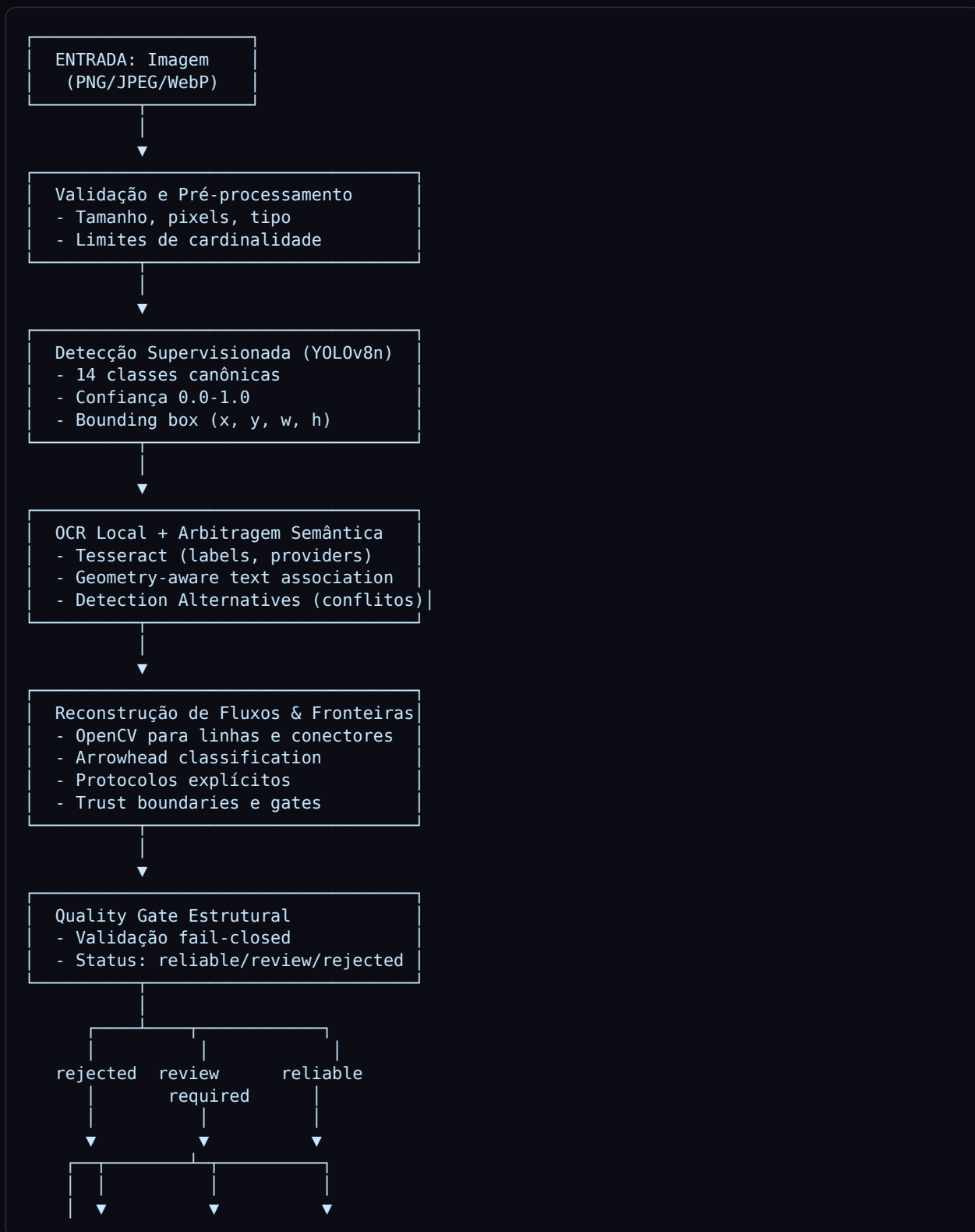
Requisito do Edital	Status	Evidência e Implementação no Projeto
6. Geração de Relatório STRIDE Consolidado	✓ Atendido	Pipeline de Relatório Multicanal: A saída consolida ameaças, contramedidas e risco em três formatos: (1) JSON estruturado com grafo de arquitetura, ameaças com status de gestão e matriz de risco, (2) Markdown com seções de resumo executivo, ameaças priorizadas e plano de mitigação, (3) PDF profissional gerado por <code>backend/pdf_report.py</code> usando ReportLab. O PDF inclui: página de cobertura, tabela de risco inerente vs. residual, matriz de severidade/status, resumo executivo, 12+ ameaças principais priorizadas por risco, plano de mitigação com responsáveis e datas, rodapé com referências. A geração é determinística — não requer LLM externa por padrão (modo generativo é opcional e desabilitado por padrão). Exemplo de saída em <code>docs/sample-threat-model.md</code> com 15+ ameaças e referências anotadas.

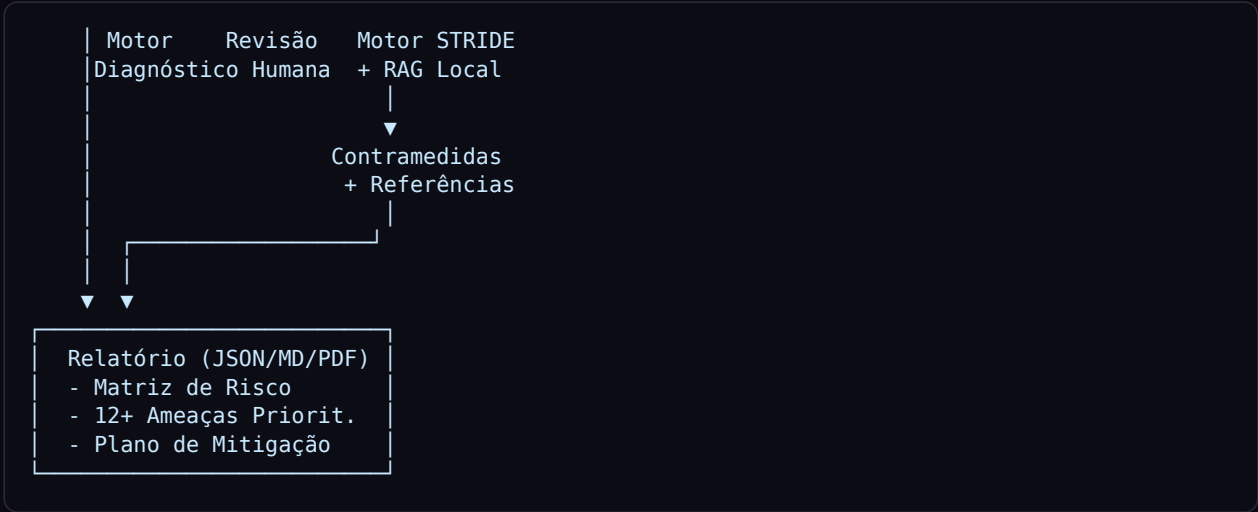
Requisito do Edital	Status	Evidência e Implementação no Projeto
7. Entregáveis e Documentação Técnica	✓ Atendido	Repositório GitHub Versionado + Documentação Completa: (a) Código-fonte 100% disponível e versionado em GitHub com histórico de commits e rastreabilidade. (b) README.md com visão geral, pré-requisitos, instrução de execução (local e Docker) e exemplos. (c) Documentação técnica detalhada em docs/architecture.md descrevendo pipeline, componentes, quality gate, STRIDE engine, RAG e segurança. (d) Documentação de dataset em docs/dataset-integration-results.md e docs/model-development.md com tabelas, origens e limitações explícitas. (e) Documentação de modelo em docs/model-evaluation-and-registration.md com métricas do detector (mAP50 0.9009, Recall 0.8417). (f) Índice operacional de demonstração em docs/submission-evidence.md com roteiro passo a passo para banca. (g) Esta documentação técnica completa pronta para exportação em PDF. (h) Vídeo de demonstração de até 15 minutos no YouTube com live demo e explicação das decisões de engenharia.

PIPELINE TÉCNICO DE IA E VISÃO COMPUTACIONAL

2.1 Arquitetura de Alto Nível

O ThreatLens implementa uma cadeia de processamento com separação clara de responsabilidades:





2.2 Detecção Supervisionada de Componentes

Responsável: `backend/detector.py`

O detector carrega o modelo YOLOv8n registrado e o executa sobre a imagem da arquitetura.

```
python

# Carregamento do modelo
model = YOLO("models/threatlens-hybrid-v2/weights/best.pt")

# Detecção
results = model.predict(source=image_path, conf=YOLO_MIN_DETECTION_CONFIDENCE)

# Processamento de resultados
for detection in results[0].boxes:
    class_id = int(detection.cls)
    confidence = float(detection.conf)
    bbox = detection.xyxy[0] # [x1, y1, x2, y2]

# Calibração de confiança (regressão logística treinada)
calibrated_conf = _calibrate_confidence(confidence, area_ratio)
```

Métricas do Detector no Teste Híbrido (39 imagens):

Métrica	Valor
Precisão	0.9297
Recall	0.8417
mAP@50	0.9009
mAP@50-95	0.8666

Saída por componente:

```
json

{
  "id": "api_1",
  "name": "API Gateway",
  "type": "api_gateway",
  "provider": "aws",
  "confidence": 0.91,
  "bbox": [120, 80, 220, 180],
```

```
"detectedBy": "yolov8",
"detectionAlternatives": []
}
```

Classes Suportadas (14 canônicas):

`api_gateway`, `backup`, `cdn`, `compute`, `database`, `identity_provider`, `internet`, `load_balancer`, `monitoring`, `queue`, `secrets_kms`, `storage`, `user`, `waf`

2.3 OCR e Reconhecimento de Contexto

Responsável: `backend/ocr.py`

O OCR local usando Tesseract extrai texto diretamente da imagem e associa geometricamente aos componentes detectados.

Fluxo:

- **Extração de texto:** Tesseract processa a imagem inteira e retorna:
 - Linhas de texto com bounding boxes
 - Confiança de cada linha
 - Posição (x, y, w, h)
- **Associação geometry-aware:** O texto é mapeado ao componente mais próximo, considerando:
 - Distância euclidiana
 - Alinhamento vertical/horizontal
 - Âncora visual preferida (abaixo de ícone, dentro de retângulo, etc.)
- **Arbitragem Semântica:** Se OCR detecta tipo ou provider que conflita com a classe YOLO:
 - Conflito é registrado em `detectionAlternatives`
 - Não sobrescreve silenciosamente a classe supervisionada
 - Fica disponível para revisão humana
 - Exemplo: YOLO detecta `compute`, OCR vê "RDS" → alternativa `database` registrada

Exemplo de Saída:

```

● ● ● json
{
  "id": "db_1",
  "name": "RDS Primary",
  "type": "database",
  "provider": "aws",
  "textEvidences": [
    {
      "text": "RDS Primary",
      "confidence": 0.87,
      "bbox": [125, 185, 220, 210],
      "associationMethod": "geometry_aware_below_icon"
    }
  ],
  "detectionAlternatives": [
    {
      "type": "storage",
      "evidence": "OCR detected 'S3 Bucket' nearby",
      "confidence": 0.42,
      "supersededBy": "database"
    }
  ]
}

```

2.4 Engine de Qualidade Estrutural (Fail-Closed & Quality Gate)

Responsável: `backend/analysis_quality.py`

O Quality Gate é acionado após reconstrução e **antes** do cálculo de STRIDE. Seu objetivo é evitar que análises baseadas em diagramas estruturalmente inválidos geguem produzir ameaças enganosas.

Critérios de Validação:

Controle	Descrição	Ação se Falhar
C01: Duplicação de componentes	Componentes com mesmo id, nome e tipo em posições diferentes (ex: diagramas comparativos AWS vs. Azure lado a lado)	Sinaliza <code>review_required</code> ou <code>rejected</code>
C02: Alinhamento suspeito	Pares de componentes idênticos alinhados perfeitamente (indicador de comparação não intencional)	Alerta com motivo e coordenadas
C03: Agrupamentos como componentes	Labels de "região", "subnet", "cluster" classificadas como componentes isolados	Marcar para revisão
C04: Inconsistência de provider	Provider detectado (ex: <code>aws</code>) conflita com evidência textual dominante (ex: "Azure")	Sinaliza divergência
C05: Self-loops sem confirmação	Componente conectado a si mesmo sem evidência de ciclo explícito	Alerta de possível erro de OCR
C06: Arestas duplicadas	Múltiplos fluxos idênticos entre os mesmos componentes	Detecta e consolida ou marca como erro

Controle	Descrição	Ação se Falhar
C07: Fronteiras cruzadas anômalas	Trust boundary que engloba ou corta através de componentes de forma incoerente	Sinaliza para revisão

Estados de Saída:

● ● ● json

```
{
  "status": "review_required", // reliable | review_required | rejected
  "score": 0.82, // 0.0-1.0
  "reasons": [
    "Possible comparative architecture detected: components 'api_1' and 'api_2' are spatially identical.",
    "Provider inconsistency: YOLO detected 'aws' but OCR found 'Azure' in labels."
  ],
  "recommendedAction": "Review the flagged items before accepting the report."
}
```

Comportamento por Status:

Status	STRIDE	Risco	Relatório	PDF
reliable	✓ Gerado	✓ Calculado	✓ Completo	✓ Exportável
review_required	✓ Gerado com alertas	✓ Calculado	✓ Com avisos	✓ Exportável
rejected	✗ Não	✗ Não	✗ Não	✗ Bloqueado

Quando **rejected**:

- Arquitetura detectada permanece acessível (diagnóstico bruto)
- JSON pode ser baixado (para análise interna)
- STRIDE, ameaças, risco e PDF são bloqueados
- **reportSuppressed: true** no response

Essa abordagem **fail-closed** é uma decisão deliberada de segurança: melhor não gerar um relatório enganoso do que gerar um falso positivo de segurança.

MAPEAMENTO STRIDE E GERAÇÃO DE CONTRAMEDIDAS

3.1 Catálogo Determinístico de Ameaças

Responsável: `backend/stride_engine.py`

O motor STRIDE não usa LLM para gerar ameaças. Todas as regras são explícitas e rastreáveis. Para cada tipo de componente e padrão arquitetural, há um conjunto de ameaças conhecidas com contramedidas associadas.

Estrutura de uma Ameaça:

```
python
{
  "stride": "Spoofing",                # Categoria STRIDE
  "title": "User identity can be impersonated",
  "severity": "Medium",                # Critical | High | Medium | Low
  "componentId": "user_1",
  "evidence": "User node detected in architecture without explicit MFA requirement.",
  "countermeasures": [
    "Require strong authentication with MFA for privileged flows.",
    "Use short-lived sessions and secure cookie attributes.",
    "Bind sensitive actions to re-authentication."
  ],
  "confidence": 0.85,                  # Baseado em detecção + provider
  "securityReferences": [
    {
      "framework": "CWE",
      "id": "CWE-287",
      "title": "Improper Authentication",
      "url": "https://cwe.mitre.org/data/definitions/287.html"
    },
    {
      "framework": "OWASP",
      "id": "A07:2021",
      "title": "Identification and Authentication Failures",
      "url": "https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/"
    }
  ],
  // ... CAPEC, MITRE ATT&CK ...
},
"management": {
  "status": "open",                   // open | mitigated | accepted | false_positive
  "owner": null,                       // Nome do responsável
  "justification": null,               // Texto da decisão
  "selectedCountermeasure": null,      // Qual contramedida foi escolhida
  "mitigationDate": null,              // Quando será mitigada
  "residualRisk": null                 // Risco após mitigação
}
}
```

3.2 Regras por Tipo de Componente

Exemplo 1: Componente `api_gateway`

```
Ameaça: Spoofing
├─ Evidência: API endpoint detectado sem autenticação explícita
├─ Severidade: High
├─ Contramedidas:
│   ├─ Validar JWT issuer, audience, expiration, signature
│   ├─ Usar OAuth2/OIDC para confiabilidade
│   ├─ Implementar mTLS para service-to-service
│   └─ Least privilege scopes por endpoint
└─ Referências: CWE-287, CAPEC-115, OWASP A07:2021, MITRE T1078
```

```
Ameaça: Tampering
├─ Evidência: Payload pode ser alterado antes de chegar ao backend
```

- └─ Severidade: High
- └─ Contramedidas:
 - └─ Validar schema na gateway e no backend
 - └─ Rejeitar campos inesperados
 - └─ Enforçar content-type restrictions
- └─ Referências: CWE-20, CAPEC-153, OWASP A08:2021, MITRE T1565

Ameaça: Denial of Service

- └─ Evidência: Endpoint exposto pode ser sobrecarregado
- └─ Severidade: High
- └─ Contramedidas:
 - └─ Throttling e quotas
 - └─ Circuit breakers
 - └─ Request size limits
 - └─ Alertas de latência e volume anômalo
- └─ Referências: CWE-400, CAPEC-125, OWASP API4:2023, MITRE T1499

Exemplo 2: Componente **database**

Ameaça: Information Disclosure

- └─ Evidência: Banco de dados pode expor registros sensíveis
- └─ Severidade: Critical Δ
- └─ Contramedidas:
 - └─ Criptografia em repouso com chaves gerenciadas
 - └─ Acesso de rede restrito
 - └─ Least privilege em roles do banco
 - └─ Masking ou tokenização de dados sensíveis
 - └─ Auditoria de acesso
- └─ Referências: CWE-200, CAPEC-118, OWASP A02:2021, MITRE T1530

Ameaça: Tampering

- └─ Evidência: Dados podem ser alterados sem controles de integridade
- └─ Severidade: High
- └─ Contramedidas:
 - └─ Transações e constraints
 - └─ Audit trail para tabelas sensíveis
 - └─ Change history
 - └─ Monitoramento de volume de writes anômalo
- └─ Referências: CWE-20, CAPEC-153, OWASP A08:2021, MITRE T1565

3.3 Matriz de Risco Inerente vs. Residual

Após STRIDE, o motor calcula dois scores:

Risco Inerente (antes de mitigação):

Fórmula: $\text{weighted_sum}(\text{severity_weight}[\text{ameaça}] * \text{confiança}[\text{ameaça}]) / \text{total_ameaças}$
 Escala: 0-100 (Low / Medium / High / Critical)

Risco Residual (após decisões de mitigação):

Fórmula: $\text{weighted_sum}(\text{severity_weight}[\text{ameaça}] * \text{confiança}[\text{ameaça}] * \text{factor_mitigação}) / \text{total_ameaças}$
 Factor de mitigação: 1.0 (open) | 0.5 (mitigated) | 0.1 (accepted)

Exemplo:

Ameaça	Severidade	Confiança	Fator Inerente	Fator Residual
API sem autenticação	High (8)	0.9	7.2	3.6 (mitigated)
DB sem backup	High (8)	0.7	5.6	0.56 (accepted)
Log sem imutabilidade	Medium (5)	0.6	3.0	3.0 (open)
Total			15.8	7.16

Interpretação:

- Risco inerente: 15.8 → **High**
- Risco residual: 7.16 → **Medium**
- Redução: 55% de mitigação implementada

3.4 Rastreabilidade de Referências de Segurança

Toda ameaça conecta-se a bases de segurança externas:

- **CWE (Common Weakness Enumeration):** Fraqueza de software subjacente
- **CAPEC (Common Attack Pattern Expression Language):** Padrão de ataque que explora a fraqueza
- **OWASP Top 10 2021:** Top vulnerabilities em aplicações web e APIs
- **MITRE ATT&CK:** Tática e técnica de ataque conhecida

Exemplo completo:

```
Ameaça: "Spoofing – User identity can be impersonated"

Rastreabilidade:
├─ Componente: user_1 (tipo: user)
├─ Fluxo relacionado: user_1 → api_1 (sem TLS)
├─ Evidência: Usuário detectado comunicando com API sem evidência de autenticação
├─ Referências:
│   ├── CWE-287 → Improper Authentication
│   │   └─ URL: https://cwe.mitre.org/data/definitions/287.html
│   ├── CAPEC-115 → Authentication Bypass
│   │   └─ URL: https://capec.mitre.org/data/definitions/115.html
│   ├── OWASP-A07:2021 → Identification and Authentication Failures
│   │   └─ URL: https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/
│   └─ MITRE-T1078 → Valid Accounts
│       └─ URL: https://attack.mitre.org/techniques/T1078/
└─ Contramedidas com mapeamento direto a esses controles
```

4.1 Formato de Saída: Multicanal

Após STRIDE e RAG, a análise é exportável em três formatos:

4.1.1 JSON Estruturado

Contém grafo completo de arquitetura, todas as ameaças com status de gestão, matriz de risco e metadados de qualidade.

● ● ● json

```
{
  "analysisId": "tl-2026-07-22-abc123",
  "timestamp": "2026-07-22T14:30:00Z",
  "analysisQuality": {
    "status": "review_required",
    "score": 0.82,
    "reasons": ["Possible comparative architecture detected."]
  },
  "architecture": {
    "components": [
      {
        "id": "api_1",
        "name": "API Gateway",
        "type": "api_gateway",
        "provider": "aws",
        "confidence": 0.91,
        "bbox": [120, 80, 220, 180]
      }
    ],
    "flows": [
      {
        "id": "flow_1",
        "from": "user_1",
        "to": "api_1",
        "protocol": "HTTPS",
        "isEncrypted": true,
        "confidence": 0.84
      }
    ]
  },
  "threats": [
    {
      "id": "threat_1",
      "stride": "Spoofing",
      "title": "User identity can be impersonated",
      "severity": "Medium",
      "componentId": "api_1",
      "evidence": "API endpoint without explicit authentication.",
      "countermeasures": ["Require MFA", "Use short-lived sessions"],
      "confidence": 0.85,
      "securityReferences": [
        {
          "framework": "CWE",
          "id": "CWE-287",
          "url": "https://cwe.mitre.org/data/definitions/287.html"
        }
      ],
      "management": {
        "status": "open",
        "owner": null,
        "justification": null
      }
    }
  ],
  "score": {
    "value": 65,
    "label": "High",
    "summary": "Architecture has significant security gaps requiring immediate mitigation."
  },
  "riskComparison": {
    "inherent": { "value": 75, "label": "High" },
    "residual": { "value": 42, "label": "Medium" },
    "reduction": 44
  }
}
```

4.1.2 Markdown Estruturado

Versão legível para documentação e integração em wikis:

● ● ● markdown

Threat Analysis Report

Architecture Overview

Metric	Value
-----	----

```
| Components      | 12 |  
| Flows           | 18 |  
| Trust Boundaries | 3  |
```

Executive Summary

The analyzed architecture contains **18 threats**, with **8 Critical/High severity**. Immediate actions:

1. Implement authentication on exposed APIs
2. Enable encryption on sensitive data flows
3. Configure database backups and monitoring

Threats Prioritized by Risk

1. Database Information Disclosure (Critical)

- **Component:** database_1
- **Severity:** Critical
- **Evidence:** Database detected without encryption at rest
- **Countermeasures:**
 - Enable encryption with managed keys
 - Restrict network access
 - Apply least privilege roles
- **References:** CWE-200, OWASP-A02:2021

[... 11+ ameaças ordenadas por risco ...]

4.1.3 PDF Professional

Gerado por `backend/pdf_report.py` usando ReportLab. Inclui:

1. **Página de Cobertura:** Logo, data, identificador da análise
2. **Sumário Executivo:** Risco inerente vs. residual, recomendações
3. **Matriz de Risco:** Severidade x Status (open/mitigated/accepted/false_positive)
4. **12+ Ameaças Priorizadas:** Cada uma com evidência, contramedidas e referências
5. **Plano de Mitigação:** Roadmap de ações, responsáveis, datas esperadas
6. **Apêndice:** Detalhes técnicos, componentes completos, fluxos

LIMITAÇÕES TÉCNICAS ASSUMIDAS E ENGENHARIA DE PRODUÇÃO

5.1 Domain Shift e Generalizacao

O detector foi treinado em 373 imagens de origens específicas (Kaggle + projeto). Isso não significa que funcionará perfeitamente em **todos** os diagramas de arquitetura do mundo.

Limitações documentadas:

- **Diagramas densos:** Linhas muito próximas, componentes sobrepostos ou linhas fragmentadas podem levar a:
 - Fluxos extras (falsos positivos de conectividade)
 - Fluxos ausentes (falsos negativos)
 - Ambiguidade em direção (arrowheads pequenos ou múltiplos possíveis)
- **Estilos visuais não vistos:** O modelo viu principalmente:
 - Icons: AWS, Azure, GCP, genéricos
 - Layouts: diagrama único, sem comparativas (até os holdouts)
 - Cores e fontes: padrão em ferramentas como Lucidchart, Draw.io, Visio

Risco: Novo estilo visual (ex: estilo customizado Miro, Mural ou design próprio) pode reduzir confiança.

- **Texto pequeno ou rotacionado:** OCR tem dificuldade com:
 - Labels dentro de ícones minúsculos
 - Texto rotacionado ou em ângulos estranhos
 - Baixo contraste

1. **Componentes fora da taxonomia:** Se o diagrama usar componentes não mapeados (ex: "custom middleware", "IoT gateway", "quantum processor"), o detector falhará ou classificará como a classe mais próxima.

Evidência no projeto:

```
Benchmark real (9 imagens de desenvolvimento + 6 blind):  
- F1 de fluxos: 0.4225 (estrutural) a 0.5296 (v15 com arbitragem)  
- Recall tipado: 0.6786  
- Domain shift em Kaggle Subset (9 imagens): mAP50 0.3598 (alarm)
```

5.2 Mecanismo de Proteção: Quality Gate e Abstention

Para evitar que análises ruins sejam apresentadas como fatos:

- **Quality Gate Estrutural:** Valida antes de STRIDE
 - Se **rejected**: Sem ameaças, sem PDF, sem relatório
 - Se **review_required**: Ameaças são geradas com alertas
 - Se **reliable**: Fluxo normal
- **Abstention e Confiança Calibrada:**
 - Detecções com confiança < 0.90 são abstraídas (não geram ameaças sozinhas)
 - Aparecem em **detectionAlternatives** para revisão
 - OCR conflitante também em alternativas, não substitui silenciosamente
- **Fail-Closed para PDF:**

- Se análise é **rejected**, PDF não é gerado
- Impede relatório enganoso

5.3 Transparência sobre Desenvolvimento vs. Generalização

O projeto separa claramente:

- **Desenvolvimento (development_tuning):** Resultados usados para ajuste de hiperparâmetros e seleção de estratégia
- **Holdout Prospectivo (prospective_holdout):** Resultado selado ANTES da abertura, prova de não viés
- **Holdout Blind Inicial:** 6 imagens anotadas post-hoc, não usadas para treino

Métrica de Desenvolvimento v15: F1 0.5296, Recall 0.6786 **Interpretação:** Essa métrica é compartilhada para transparência, mas **não** é apresentada como evidência de generalização universal. A metodologia está documentada em [docs/final-technical-evidence.md](#).

5.4 Configuração de Confiança e Limiares

O operador pode ajustar:

```

● ● ● bash

# Arquivo: .env
YOLO_MIN_DETECTION_CONFIDENCE=0.30      # Confiança mínima bruta do YOLO
YOLO_CONFIDENCE_THRESHOLD=0.90          # Limiar de aceitação após calibração
FLOW_STRATEGY=legacy                     # ou junction_aware_controlled

```

Comportamento:

- Se **YOLO_CONFIDENCE_THRESHOLD** é 0.90: Apenas detecções calibradas > 0.90 geram ameaças
- Se **YOLO_MIN_DETECTION_CONFIDENCE** é 0.30: Até 0.30 bruta é considerada para alternativas
- Esse design permite trade-off entre recall (mais ameaças) e precisão (menos falsos positivos)

5.5 Roadmap de Melhoria

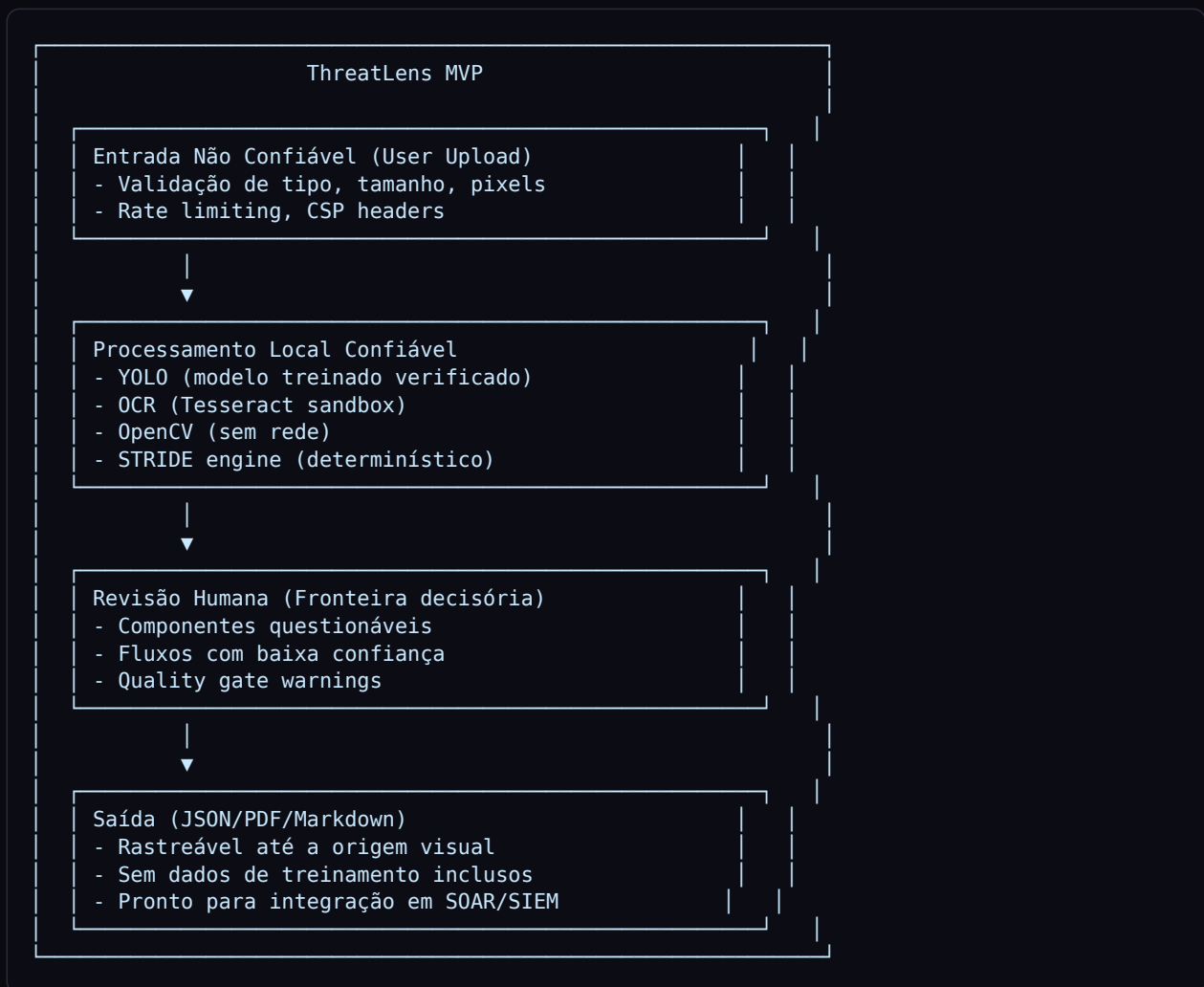
Limitações sabidas e próximas iterações:

Limitação	Estratégia de Melhoria
Domain shift em estilos novos	Active learning com mais diagramas reais
Fluxos em diagramas densos	Classificador supervisionado de arestas
Arrowhead ambíguo	Calibração separada de direção
OCR em texto pequeno	Modelo de OCR treinado ou pre-processamento adaptativo
Componentes fora da taxonomia	Extensão de classes (15+) com retraining

6.1 Princípios de Design

1. **Zero Trust na Entrada:** Toda entrada é não confiável até validação
2. **Separation of Concerns:** Cada etapa é auditável isoladamente
3. **Fail-Closed:** Melhor não gerar ameaça enganosa do que gerar uma falsa
4. **Local-First:** Sem chamadas remotas obrigatórias
5. **Transparência:** Todas as decisões são rastreáveis

6.2 Fronteiras de Confiança na Arquitetura



6.3 Serviços Remotos Opcionais

Por padrão, **todas** as chamadas remotas estão **desabilitadas**:

```
● ● ● bash
ENABLE_GENERATIVE_REPORTS=false # Sem Gemini obrigatório
ENABLE_REMOTE_VALIDATION=false # Sem Groq para validação
ENABLE_VISION_FALLBACK=false # Sem Gemini Vision fallback
```

Isso significa:

- Detector: ✓ Local
- OCR: ✓ Local (Tesseract)
- STRIDE: ✓ Local (determinístico)
- RAG: ✓ Local (embeddings + lexical fallback)
- PDF: ✓ Local (ReportLab)

Se quiser LLM generativo para enriquecer relatório:

- Deve ativar explicitamente `ENABLE_GENERATIVE_REPORTS=true` e fornecer `GEMINI_API_KEY`
- Relatório base permanece determinístico sem isso

Este documento é parte de um conjunto de entregáveis para o Hackathon:

Entregável	Localização	Formato	Propósito
Código-Fonte	GitHub	.git repository	Implementação completa e versionada
README	README.md	Markdown	Visão geral, setup e exemplos
Documentação Técnica	Este arquivo	Markdown + PDF	Detalhamento arquitetural
Documentação de Dataset	docs/dataset-*.md	Markdown	Origens, splits, anotações
Documentação de Modelo	docs/model-*.md	Markdown	Métricas, treino, limitações
Índice Operacional	docs/submission-evidencia.md	Markdown	Roteiro de demonstração para banca
Vídeo Demonstrativo	YouTube	15 min video	Live demo + explicações técnicas

O ThreatLens AI implementa um MVP robusto de modelagem automática de ameaças STRIDE. A solução:

✓ Atende todos os 7 requisitos oficiais do hackathon ✓ Implementa defesas contra alucinações (quality gate, fail-closed) ✓ Mantém rastreabilidade completa de detecção até ameaça ✓ Oferece múltiplos canais de saída (JSON, Markdown, PDF) ✓ Usa referências externas verificáveis (CWE, CAPEC, OWASP, MITRE) ✓ Transparência sobre limitações e domain shift ✓ Código aberto, documentação completa, reprodutível

Recomendações para uso em produção:

1. Expandir dataset com mais diagramas reais e estilos visuais
2. Implementar active learning com falsos positivos reais
3. Adicionar classificador supervisionado de existência de arestas
4. Integrar com workflow corporativo de aprovação de riscos
5. Realizar nova avaliação prospectiva com holdout ainda não observado

Links de Referência

- **CWE (Common Weakness Enumeration):** <https://cwe.mitre.org>
- **CAPEC (Common Attack Pattern Expression Language):** <https://capec.mitre.org>
- **OWASP Top 10 2021:** <https://owasp.org/Top10>
- **MITRE ATT&CK:** <https://attack.mitre.org>
- **STRIDE (Microsoft):**
<https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats>
- **YOLOv8 Documentation:** <https://docs.ultralytics.com>
- **Tesseract OCR:** <https://github.com/UB-Mannheim/tesseract>
- **FastAPI:** <https://fastapi.tiangolo.com>
- **ReportLab (PDF):** <https://www.reportlab.com/>

Documento preparado em: Julho de 2026 **Versão:** 1.0 — Entrega Final **Status:** Pronto para Exportação em PDF e Submissão à Banca

Versão: **1.0.0-mvp**. O backend FastAPI escuta por padrão em **127.0.0.1:8000**; no pacote Docker, o proxy Node expõe a API em **http://127.0.0.1:4173/api**.

GET /health

Sonda leve de processo. Não carrega o detector, não executa inferência e não abre benchmarks.

```
● ● ● json
{"status":"ok","version":"1.0.0-mvp","flowStrategy":"legacy"}
```

O objeto adicional **flowStrategies** informa estratégia selecionada, padrão e opções. **GET /ready** é a sonda profunda: verifica detector, calibração e RAG e pode retornar 503.

POST /analyze/image

Entrada **multipart/form-data**, campo obrigatório **image**. Tipos aceitos: PNG, JPEG e WebP. O limite padrão é 25 MB e 40 milhões de pixels. A resposta é a arquitetura detectada com **components**, **flows**, **trustBoundaries**, **detectionAlternatives**, **flowStrategy**, **flowStrategyTrace** e **analysisQuality**. Não gera o relatório STRIDE.

POST /analyze/json

Entrada JSON com **components**, **flows** e, opcionalmente, **trustBoundaries**, metadados e decisões de mitigação. Limites padrão: 2 MB, 200 componentes, 1.000 fluxos e 100 fronteiras. A resposta contém **architecture**, **threats**, **score**, **riskComparison**, **graph**, **coverage**, **reportMarkdown**, rastreabilidade e **pipeline**.

Qualidade estrutural

Todas as respostas de análise expõem **analysisQuality**:

```
● ● ● json
{
  "analysisQuality": {
    "status": "reliable",
    "score": 1.0,
    "reasons": [],
    "recommendedAction": "A estrutura passou pelo gate automático; mantenha a revisão humana antes da aprovação final."
  }
}
```

Os status aceitos são **reliable**, **review_required** e **rejected**. Uma rejeição impede STRIDE, RAG, risco e relatório; a resposta usa **reportSuppressed: true**, **threats: []** e **score: null**, preservando a arquitetura e as razões para diagnóstico. **/report/pdf** recusa resultados rejeitados com 422.

POST /analyze/full

Entrada **multipart/form-data** com **image**, **architecture_json** ou ambos. A imagem é prioritária; o JSON funciona como fallback quando a detecção falha. Executa detecção, estratégia de fluxo, STRIDE, RAG e relatório. **pipeline.flowStrategy** e **pipeline.flowStrategyTrace** registram a estratégia e as decisões aplicadas.

Estratégias

`FLOW_STRATEGY` é lida na inicialização. Sem a variável, o sistema usa `legacy`.
`junction_aware_controlled` é experimental, reversível, não redireciona endpoints e não aplica automaticamente decisões `review_only`. Valor desconhecido falha fechado com 422 quando a análise tenta aplicar a estratégia.

Códigos e erros

Código	Significado
200	Requisição concluída
400	Imagem corrompida ou payload inválido em nível de conteúdo
413	Corpo ou imagem acima do limite
415	Tipo de imagem não suportado
422	Estrutura, IDs, bbox, estratégia ou formulário inválido
503	Detector supervisionado indisponível e fallback visual desativado

Erros são JSON no formato `{"detail": "mensagem"}` e as respostas recebem `X-Request-ID` para correlação.

Escopo

O modelo supervisionado detecta componentes de arquitetura. Fluxos, direção, fronteiras, STRIDE e risco são etapas separadas; não foram usados como rótulos do detector.

Dados e origem

O baseline `dataset/hybrid_v2` reúne 300 diagramas do conjunto complementar do projeto e 73 arquiteturas independentes selecionadas do Software Architecture Dataset público no Kaggle. O inventário remoto encontrou 8.700 imagens PNG, 8.700 XMLs Pascal VOC e 852 grupos de arquiteturas originais. Foi mantida apenas uma variante por grupo Kaggle porque as demais eram augmentations da mesma arquitetura.

As anotações Kaggle em Pascal VOC foram convertidas para YOLO e mapeadas para a taxonomia canônica. Objetos fora do escopo foram preservados na auditoria, mas não convertidos em rótulos artificiais.

Classes e divisões

O conjunto final possui 373 imagens e 3.048 objetos anotados:

Split	Imagens	Labels
Treino	265	265
Validação	69	69
Teste	39	39

As 14 classes são `api_gateway`, `backup`, `cdn`, `compute`, `database`, `identity_provider`, `internet`, `load_balancer`, `monitoring`, `queue`, `secrets_kms`, `storage`, `user` e `waf`. Todas aparecem nos três splits. A auditoria não encontrou imagens ou labels órfãos nem famílias Kaggle atravessando splits.

Treinamento e registro

O detector registrado é um YOLOv8n em `models/threatlens-hybrid-v2/weights/best.pt`. A etapa final utilizou replay balanceado de 55 diagramas Kaggle e 55 diagramas da fonte anterior, validação balanceada de 9 + 9 imagens, resolução 640 e AdamW. O hash SHA-256 registrado do peso é `2aebcd0611927505d60a10d23ce11d0e5211cf2e91ca0c4394921fac006c865d`.

Métricas reais do detector

No teste híbrido de 39 imagens:

Métrica	Valor
Precisão	0,9297
Recall	0,8417
mAP50	0,9009
mAP50-95	0,8666

Na fatia de 9 diagramas Kaggle, o modelo final alcançou mAP50 0,3598 e recall 0,3951. Essa fatia é pequena e é tratada como alarme de domain shift, não como prova de cobertura universal.

Prevenção de vazamento

- A unidade de agrupamento Kaggle é a arquitetura original, não a imagem aumentada.
- Variantes da mesma arquitetura não cruzam treino, validação e teste.
- O replay usa manifestos explícitos e mantém o teste híbrido inalterado.
- Os holdouts end-to-end não foram executados durante `DELIVERY-001`.

Limitações

O dataset é pequeno, contém uma fração relevante de diagramas gerados e cobre apenas 14 classes canônicas. Ícones desconhecidos, novos estilos visuais, texto pequeno e diagramas densamente conectados podem reduzir o desempenho. As métricas do detector não medem a qualidade dos fluxos ou do relatório STRIDE.

Evidências detalhadas: `docs/dataset-integration-results.md`,
`docs/model-evaluation-and-registration.md` e
`models/threatlens-hybrid-v2/model-card.json`.

Reconstrução de fluxos em development_tuning

Estratégia	Corretos	Falsos positivos	Ausentes	Recall	F1	Acurácia de direção
legacy	52	90	19	0,7324	0,4883	0,8654
junction_aware_controlled	52	75	19	0,7324	0,5253	0,8654

A configuração controlada corresponde a `full_without_endpoint_redirect + structural_line_gate`. Ela bloqueou 15 falsos fluxos, sem bloquear verdadeiros positivos na amostra de desenvolvimento, e preservou os controles humanos C01-C07.

Interpretação correta

Os números acima foram obtidos somente no split `development_tuning` e com apoio das revisões humanas da TL-004. Eles validam integração controlada e reversível, mas não são evidência de generalização. A estratégia permanece opt-in e `legacy` continua padrão.

O detector de componentes tem avaliação separada no teste híbrido de 39 imagens: precisão 0,9297, recall 0,8417, mAP50 0,9009 e mAP50-95 0,8666. Essas métricas não devem ser confundidas com F1 de fluxos ou F1 de ameaças.

Evidências preservadas

- Gate v15: F1 0,5296, precisão 0,4343, recall 0,6786, 152 ameaças corretas e 198 extras.
- Auditoria prospectiva v12: cadeia hash-selada verificada separadamente.
- Snapshot `legacy`: 142 fluxos no contrato de promoção, mantido imutável.
- `DELIVERY-001`: não executa `blind_holdout` nem `prospective_holdout`.

Os resultados históricos permanecem em `data/results/`; o manifesto de reprodutibilidade registra hashes, versões e seeds.

1. Objetivo técnico

O ThreatLens AI implementa um pipeline de modelagem de ameaças assistida por IA. A entrada é uma imagem de arquitetura; a saída é uma representação estruturada acompanhada de ameaças STRIDE, contramedidas, referências de segurança e relatório exportável.

A solução separa cinco responsabilidades:

1. detecção visual de componentes;
2. reconstrução de conectividade e fronteiras;
3. avaliação de qualidade estrutural;
4. inferência determinística de ameaças;
5. recuperação de conhecimento e geração do relatório.

Essa divisão reduz alucinações e permite testar e auditar cada etapa isoladamente.

2. Fluxo principal

```
● ● ● text

Imagem
-> validação e pré-processamento
-> detecção supervisionada de componentes
-> OCR e evidências visuais
-> reconstrução de fluxos e fronteiras
-> quality gate estrutural
-> representação JSON e grafo
-> revisão humana
-> motor STRIDE
-> RAG local de vulnerabilidades e controles
-> contramedidas, referências e relatório
```

3. Entrada e validação

`backend/main.py` expõe a API FastAPI. A camada de entrada valida tipo, bytes, quantidade de pixels, cardinalidade e estrutura dos payloads. PNG, JPEG e WebP são aceitos; entradas inválidas falham antes da inferência.

O middleware gera um `X-Request-ID` por requisição. O proxy em `server.mjs` publica a API sob `/api`, serve o frontend e aplica Content Security Policy e demais cabeçalhos de segurança.

Serviço	Porta padrão	Responsabilidade
FastAPI	8000	detecção, análise, health, readiness e PDF
Node.js	4173	frontend estático e proxy <code>/api</code>

`GET /health` é uma sonda leve. `GET /ready` valida detector, calibração e base RAG.

4. Pipeline de Visão Computacional

4.1 Detecção supervisionada

`backend/detector.py` carrega o peso registrado em `models/threatlens-hybrid-v2/weights/best.pt`. O YOLOv8n detecta 14 classes canônicas e retorna classe, confiança e bounding box.

```
● ● ● json
{
  "id": "api_1",
  "name": "API Gateway",
  "type": "api_gateway",
  "provider": "aws",
  "confidence": 0.91,
  "bbox": [120, 80, 220, 180]
}
```

As confianças passam pela calibração registrada. Providers são atribuídos somente quando há evidência visual ou textual suficiente.

4.2 OCR e arbitragem semântica

O Tesseract local extrai labels, providers e protocolos explícitos. A geometria associa o texto ao componente mais compatível, considerando distância, posição e âncora visual.

OCR não sobrescreve silenciosamente a classe supervisionada. Conflitos são registrados em `detectionAlternatives`, incluindo tipo, provider, confiança, bbox, metadados e `supersededBy`. Essas alternativas permanecem fora de componentes ativos, grafo, STRIDE e risco.

4.3 Reconstrução de estrutura

`backend/diagram_structure.py` usa OpenCV para extrair segmentos, linhas e regiões retangulares. A etapa reconstrói:

- linhas fragmentadas e cotovelos;
- associação de endpoints a componentes;
- direção por arrowheads;
- protocolos explícitos próximos aos conectores;
- fronteiras de confiança candidatas;
- cruzamentos, bifurcações e troncos compartilhados;
- barreiras e portas de componentes;
- atalhos transitivos e linhas estruturais.

`backend/flow_strategy.py` permite selecionar:

Estratégia	Comportamento
legacy	Padrão estável e baseline preservada.
junction_aware_controlled	Estratégia experimental e reversível com validações junction-aware.

A estratégia controlada combina `full_without_endpoint_redirect` com `structural_line_gate`. Ela não reativa redirecionamento automático de endpoints e não aplica decisões `review_only` como fatos.

No conjunto `development_tuning`, a estratégia preservou 52 conexões corretas e reduziu falsos fluxos de 90 para 75, elevando o F1 de 0,4883 para 0,5253 sem perda de recall.

5. Quality gate estrutural

`backend/analysis_quality.py` atua depois da reconstrução e antes do STRIDE. O objetivo é evitar que risco e ameaças sejam calculados sobre uma arquitetura inconsistente.

O gate avalia:

- componentes com nome e tipo duplicados;
- pares duplicados espacialmente alinhados, indicando diagramas comparativos;
- labels de contas, regiões ou agrupamentos classificados como componentes;
- provider incompatível com evidência textual dominante;
- self-loop sem confirmação explícita;
- arestas direcionadas duplicadas.

Status	Decisão da pipeline
reliable	STRIDE e relatório podem prosseguir.
review_required	Resultado segue com alertas e revisão obrigatória.
rejected	STRIDE, RAG, risco, relatório e PDF são suprimidos.

Em uma rejeição, a arquitetura detectada permanece disponível para diagnóstico e download JSON, mas `threats` é vazio, `score` é nulo e `reportSuppressed` é verdadeiro.

6. Grafo e revisão humana

Após a normalização, componentes ativos tornam-se nós e fluxos aceitos tornam-se arestas. Metadados de provider, protocolo, confiança, evidências e fronteiras acompanham o grafo.

A revisão humana é a fronteira entre inferência visual e decisão de segurança. Componentes e fluxos pendentes são apresentados na interface com confiança e motivo da revisão.

7. Motor STRIDE e contramedidas

`backend/stride_engine.py` aplica regras determinísticas por tipo de componente, provider, fluxo, protocolo e transição de fronteira.

Evidência arquitetural	STRIDE relacionado	Exemplos de contramedidas
API exposta sem autenticação explícita	Spoofing	MFA, identidade federada e validação de token
Fluxo sensível sem protocolo seguro	Information Disclosure	TLS, mTLS e gestão de certificados
Banco sem backup explícito	Denial of Service	backups, RTO/RPO e testes de restauração
Chave sem controle de acesso identificado	Elevation of Privilege	rotação, menor privilégio e auditoria
Operação sem trilha de auditoria	Repudiation	logs imutáveis e correlação de eventos

Cada ameaça registra categoria STRIDE, severidade, confiança, evidência, componente ou fluxo relacionado, contramedidas e referências como CWE, CAPEC, OWASP e MITRE ATT&CK, quando aplicáveis.

O motor também mantém status de gestão da ameaça, responsável, justificativa, mitigação escolhida e comparação entre risco inicial e residual.

8. RAG e geração de relatório

`backend/rag.py` consulta documentos versionados em `backend/knowledge`. A recuperação é orientada por componentes e categorias STRIDE, com embeddings locais quando disponíveis e fallback lexical determinístico.

O caminho padrão não exige cloud nem custos externos. Enriquecimento generativo e validação remota são opt-in. Mesmo quando habilitada, a camada generativa não pode criar componentes ou fluxos ausentes da arquitetura analisada.

`backend/pdf_report.py` gera o PDF nativo. A interface também permite baixar o JSON completo e imprimir a visualização executiva.

9. Segurança e fronteiras de confiança

- upload para API: entrada não confiável, limitada e validada;
- imagem para modelo e OCR: conteúdo não confiável processado localmente;
- detecção para revisão: hipótese técnica, não fato confirmado;
- arquitetura revisada para STRIDE: principal fronteira decisória;
- serviços remotos opcionais: desabilitados por padrão e dependentes de opt-in explícito.

10. Reprodutibilidade e avaliação

O projeto registra hashes do modelo, datasets, configurações, código e artefatos no manifesto `data/manifests/reproducibility.json`.

Controles disponíveis:

- gate automático da baseline v15;
- auditoria hash-selada da cadeia prospectiva v12;
- benchmark real com AWS, Azure, GCP, diagramas genéricos e estilos FIAP;
- benchmark estrutural e golden set STRIDE;
- testes unitários, integração, frontend e regressão;
- 350 testes aprovados e verificador global em estado **PASS**.

As métricas do detector, de fluxos e de ameaças são tratadas separadamente. Resultados de desenvolvimento não são apresentados como evidência cega de generalização.

11. Limitações atuais

- diagramas densos ainda podem produzir fluxos extras ou ausentes;
- arrowheads pequenos, borrados ou sobrepostos reduzem a confiança de direção;
- bordas, grades e elementos internos de ícones podem parecer conectores;
- o detector cobre 14 classes canônicas;
- a diversidade de diagramas reais ainda é limitada;
- réplicas perfeitamente alinhadas podem exigir revisão do quality gate;
- as métricas do detector não medem diretamente a qualidade do relatório STRIDE;
- o resultado é assistência técnica e requer validação humana.

12. Trabalhos futuros

- active learning com falsos positivos reais;
- classificador supervisionado de existência de arestas;

- calibração separada de OCR, existência de fluxo e direção;
- maior cobertura de diagramas reais e estilos visuais;
- OCR para labels pequenos, rotacionados e de baixo contraste;
- fronteiras de confiança hierárquicas;
- integração com backlog corporativo de riscos e workflow de aprovação;
- nova avaliação prospectiva com diagramas ainda não observados.

Evidências para Entrega e Demonstração

Este documento é o índice operacional da entrega. Ele aponta para evidências já produzidas e evita executar treinamento, holdouts ou Docker durante a apresentação.

Mensagem central

O ThreatLens não gera ameaças diretamente da imagem. Ele detecta componentes com um modelo supervisionado, reconstrói uma arquitetura revisável, aplica um gate de qualidade e somente então executa regras STRIDE e recuperação local de contramedidas.

Matriz dos requisitos

Requisito do hackathon	Evidência demonstrável
Interpretar um diagrama em imagem	<code>data/sample-diagrams/02-mixed-components.jpg</code> e dashboard
Identificar componentes	boxes, classes, provider, confiança e <code>bbox</code> na resposta da API
Dataset de arquiteturas	<code>docs/dataset-integration-results.md</code> e manifestos em <code>data/manifests/</code>
Dataset anotado	373 imagens, 3.048 objetos e 14 classes documentados no <code>README.md</code>
Modelo supervisionado treinado	<code>docs/model-development.md</code> e model card registrado
Relatório STRIDE	<code>docs/sample-threat-model.md</code> e JSON de amostra
Vulnerabilidades e contramedidas	referências CWE, CAPEC, OWASP e MITRE ATT&CK no dashboard e PDF
Documentação do fluxo	<code>docs/architecture.md</code>
Mais de uma visualização	dashboard, JSON, Markdown e PDF

Evidências quantitativas oficiais

Detector supervisionado

Métrica	Valor
Precisão	0,9297
Recall	0,8417
mAP@50	0,9009
mAP@50-95	0,8666

Fonte: model card e resultados registrados de `threatlens-hybrid-v2`.

Gate de regressão v15

Métrica	Valor
F1	0,5296
Precisão	0,4343
Recall	0,6786
Ameaças corretas	152
Falsos positivos	198

Evidência: `data/results/v15-regression-gate/`. Esses valores pertencem ao split `development_tuning`; não devem ser apresentados como novo resultado cego.

Estratégia junction-aware controlada

Métrica	Valor
Conexões corretas	52
Falsos fluxos	75
Fluxos ausentes	19
Recall	0,7324
F1	0,5253
Acurácia de direção	0,8654
Verdadeiros positivos bloqueados	0

Evidências: `data/results/tl004f-integrated-ablation/` e `data/results/tl-struct-001a-final-gate/`. A estratégia é experimental, opt-in e validada somente em desenvolvimento; `legacy` permanece o padrão.

Qualidade e engenharia

- 350 testes automatizados aprovados.
- Verificador global: `PASS`.
- Auditoria prospectiva v12: `PASS`.
- Controles C01-C07: `PASS`.
- Manifesto com hashes em `data/manifests/reproducibility.json`.

Os 85 testes encontrados em artefatos históricos são a baseline mínima preservada pela TL-001, não o total atual da suíte.

Rota de demonstração sem Docker

Preparação

Na raiz do projeto:

```
● ● ● powershell
.venv\Scripts\python.exe -c "import cv2, fastapi, ultralytics; print('Dependências OK!')"
npm.cmd run verify
npm.cmd run report:sample-pdf
npm.cmd run dev
```

Abra `http://127.0.0.1:4173` e mantenha o terminal aberto.

Caso principal

- 1. Selecione `data/sample-diagrams/02-mixed-components.jpg`.
- 2. Mostre componentes detectados, confianças e bounding boxes.
- 3. Mostre fluxos, protocolos, fronteiras e itens pendentes de revisão.
- 4. Confirme a arquitetura revisada.
- 5. Gere STRIDE e abra a rastreabilidade de uma ameaça.
- 6. Mostre regra, componente ou fluxo de origem, fonte RAG e contramedida.
- 7. Exporte JSON, Markdown e PDF.

Caso de segurança fail-closed

- 1. Envie `E:\teste_aws.png`, se disponível.
- 2. Mostre que o sistema identifica uma entrada comparativa ou estruturalmente inconsistente.
- 3. Destaque o status `rejected`, as razões e a ação recomendada.
- 4. Confirme que STRIDE, risco, contramedidas e PDF foram bloqueados.

Evidência de contingência: `data/results/mvp-hardening-001/external-image-quality.json`.

Plano de contingência

Falha durante a apresentação	Ação
Docker indisponível	Use <code>npm.cmd run dev</code> ; Docker não faz parte da rota necessária.
Detector demora	Use <code>data/results/web-mvp-001/sample-analysis-report.json</code> .
Backend não inicia	Mostre <code>docs/sample-threat-model.md</code> e o PDF previamente gerado.
Arquivo externo indisponível	Abra <code>data/results/mvp-hardening-001/external-image-quality.json</code> .
Sem internet	Continue normalmente; detector, RAG e relatório base são locais.
Pergunta sobre generalização	Diferencie desenvolvimento, holdouts selados e resultados pós-hoc.

Evidências para deixar abertas antes da banca

- 1. Dashboard em `http://127.0.0.1:4173`.
- 2. `docs/sample-threat-model.md`.
- 3. `data/results/web-mvp-001/sample-analysis-report.json`.
- 4. `data/results/mvp-hardening-001/external-image-quality.json`.
- 5. `data/results/v15-regression-gate/` com o relatório mais recente.
- 6. `data/manifests/reproducibility.json`.
- 7. `output/pdf/threatlens-sample-report.pdf`.

Frases de defesa técnica

- O supervisionado detecta componentes; o STRIDE é aplicado sobre o grafo revisado.
- A estratégia experimental não substituiu silenciosamente a baseline legacy.
- Resultado de desenvolvimento não é apresentado como generalização.
- Uma análise rejeitada preserva o diagnóstico, mas não produz risco ou ameaças enganosas.
- Ausência de alerta não significa ausência de risco; o produto apoia, mas não substitui revisão de segurança.

O que não executar na apresentação

- treinamento ou fine-tuning;
- `blind_holdout` ou `prospective_holdout`;
- atualização de baseline ou manifesto;
- download de modelos ou datasets;
- alteração de thresholds ou de `FLOW_STRATEGY`;
- instalação ou diagnóstico do Docker.

Critério de prontidão

A demonstração está pronta quando o frontend abre, `/api/ready` responde com sucesso, o caso de amostra gera análise, o PDF de contingência existe e os quatro artefatos principais deste índice podem ser abertos sem rede.

- Diagramas densos: cruzamentos, linhas longas, cotovelos, bordas de grupos e conectores paralelos ainda podem produzir fluxos extras ou ausentes.
- Setas: arrowheads pequenos, borrados ou sobrepostos reduzem a confiança da direção.
- Endpoints: proximidade geométrica pode associar uma linha ao componente incorreto.
- `review_only`: sinais incompletos são apresentados para revisão e nunca bloqueados automaticamente pela estratégia controlada.
- Caso C04: a junção densa supervisionada exige proteção explícita e não sustenta uma regra geral de redirecionamento de endpoints.
- Revisão humana: componentes pendentes, conflitos OCR, fluxos inferidos, protocolos desconhecidos e fronteiras candidatas devem ser confirmados antes de decisões reais.
- Generalização: a estratégia controlada foi validada apenas em `development_tuning`; nenhum holdout foi executado em `DELIVERY-001`.
- Estratégia padrão: `junction_aware_controlled` é experimental e opt-in. `legacy` permanece padrão e reversível por configuração.
- Taxonomia: o detector cobre 14 classes canônicas; serviços fora dessas classes são aproximados, absteridos ou encaminhados para revisão.
- Segurança: o relatório modela ameaças, mas não substitui teste de invasão, revisão de configuração cloud, análise de código ou validação de controles em produção.
- RAG e LLM: o modo local é determinístico. Recursos remotos são opcionais, exigem configuração explícita e não devem receber diagramas confidenciais sem autorização.

Links de Referência

- **CWE (Common Weakness Enumeration):** <https://cwe.mitre.org>
- **CAPEC (Common Attack Pattern Expression Language):** <https://capec.mitre.org>
- **OWASP Top 10 2021:** <https://owasp.org/Top10>
- **MITRE ATT&CK:** <https://attack.mitre.org>
- **STRIDE (Microsoft):**
<https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats>
- **YOLOv8 Documentation:** <https://docs.ultralytics.com>
- **Tesseract OCR:** <https://github.com/UB-Mannheim/tesseract>
- **FastAPI:** <https://fastapi.tiangolo.com>
- **ReportLab (PDF):** <https://www.reportlab.com/>

Documento preparado em: Julho de 2026 **Versão:** 1.0 — Entrega Final **Status:** Pronto para Exportação em PDF e Submissão à Banca